ETH Zürich

Automatic Control Laboratory (IfA)

P&S Mission Impossible Cartpole 4.0



Designed by Niklas Schmid, Nathan Soldati & Prof. John Lygeros

Instruction Sheet

What this course is about.

It's about learning how to design controllers for real hardware. To complete the course, you are only expected to

- derive a model of the system dynamics and build your own simulator (you will need it).
- implement a controller that stabilizes the pole upright (the real one, not your simulated pole).
- implement a controller that swings the pole up (the real one, not your simulated pole).

We don't care how you do it, and you are welcome to do much more than that. You can try out classical approaches, implement brand-new research, or hack your way through the course with your own heuristics. You can use large language models, or derive everything yourself. This is your chance to apply all the tools you studied in lectures. You will learn what it takes run control algorithms on real systems, and understand their strengths and weaknesses. We will assist you on your way.



In control engineering, you always validate controllers in simulation before running them on hardware (it is much harder to debug on hardware!). You may only expect your controller to be safe and functional in the real world when it works well in your simplified simulation. We strongly recommend to follow this workflow. Wisely invest your energy, time and sanity.

We aim to continuously improve this course. Please share your feedback with us, either anonymously via this form or by sharing your thoughts with the TA's during the course.

Figure 1: Scan this code to enter the anonymous feedback form.

CONFIDENTIAL: Project CartPole4.0

Dear Engineer,

Please treat this message as confidential. Our progress on the new CartPole4.0 project is advancing rapidly. Two weeks ago, our biggest competitor, GogoPendulum, successfully stabilized their cart-pole prototype (according to our insiders). We must outperform them and bring our system to market before they do.

Weve hired the best control engineersnamely, youto reclaim the lead and drive our product to success. The pole must swing up as quickly as possible and stabilize in the upright position. Our electrical engineers have assembled a first prototype, including a software interface for your use. Youll find it in our basement (R&D D-floor, secret room D10.1).

Our CTO has attached materials outlining the initial steps. Were counting on you!

International Feedback Association (IfA)

The CartPole4.0 Prototype

The cart pole system consists of two elements: a cart moving horizontally along a track and a pole mounted on it with the pivot at its base, see Fig. 2.

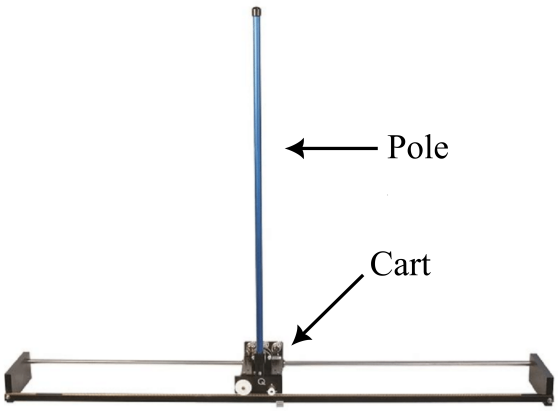


Figure 2: Cart-pole system at IfA.

A servo motor is mounted on the cart to move it horizontally along the track. The motor force is the only input to our system. By moving the cart, it is possible to swing the pole up and stabilize it in the upright vertical position. Sensors acquire the cart's horizontal position and the pole's rotation angle. The variables and parameters used to describe the system dynamics are listed in table 1 and 2.

Variables	Description
F_c	Force generated by the cart's motors
$u = V_m$	Input voltage V_m to the motors, which we declare as input u
x_c, y_c	Cart position
x_p, y_p	Position of the center of mass of the pole
α	Pole angle in rad (upright is zero, increasing counter-clockwise)

Table 1: Overview of the system variables, see Fig. 3.

Parameter	**Description**	**Value**
g	Gravitational Acceleration	9.81 $\mathrm{m/s^2}$
m_p	Pole Mass	0.127 kg
m_c	Cart Mass	0.57 kg
r	Distance from Pivot to Center of Mass	0.1778 m
B_f	Friction Coefficient	unknown $\mathrm{N \cdot s/m}$
K_g	Planetary gearbox gear ratio	3.71
K_m	Motor back-EMF constant	0.00767 $\mathrm{V \cdot s/rad}$
K_t	Motor torque constant ($K_t = K_m$)	0.00767 $\mathrm{N \cdot m/A}$
J_m	Motor moment of inertia	3.9e−7 $\mathrm{kg \cdot m^2}$
r_{mp}	Motor pinion radius	6.35e−3 m
R_m	Motor armature resistance	2.6 Ω

Table 2: Overview of the system parameters. They are already implemented in the file *system_setup.m*.

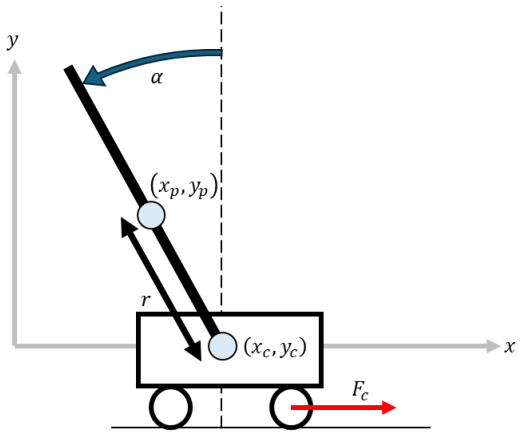


Figure 3: Variables of the cart-pole system. α: Pole angle, x_c, y_c: Cart position, x_p, y_p: position of the cart center of mass, F_c: Force generated by the motors, r: Distance from the pivot to the pole center of mass.

STEP 1

MODELING

Our CTO recommends to start by modelling the system and analysing its dynamics. This step is crucial to program your own simulator on which you validate your controllers.

Let x be the position of a free object and denote the velocity and acceleration by $\frac{\partial x}{\partial t} = \dot{x}$, $\frac{\partial^2 x}{\partial^2 t} = \ddot{x}$. By Newton's second law, translational motion of a free body is induced by forces,

$$m\ddot{x} = \sum_i F_i \,, \tag{1}$$

where m is the mass of the rigid body, $\ddot{x}$ is its acceleration and the right-hand side of the equation represents the sum of all the forces acting on it. For rotational motion, Newton's law is written as

$$J\ddot{\alpha} = \sum_i M_i \,, \tag{2}$$

where J is the moment of inertia of the rigid body, $\ddot{\alpha}$ is its angular acceleration and the right-hand side of the equation represents the sum of all the momenta acting on it in the rotational direction α.

Forces F that do not act on the center of mass of an object but at a distance r generate a moment

$$M = F \times r,$$

which yields $|M| = |F| r \sin(\gamma)$, where γ is the angle between the force direction and the vector connecting the center of mass to the point on which the force is acting (Fig. 4). The moment of inertia J for a pole with mass m and length l (not radius!) with its pivot at its end is $J = \frac{1}{3} m l^2$.

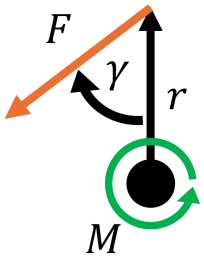


Figure 4: The force F is acting at a distance $|r|$ from the center of mass (black circle). The force generates a moment of magnitude $|M| = |F| r \sin(\gamma)$, where γ is the angle between F and r.

The input to the system is the voltage V_m of the servo motor, which results in the force

$$F_c = \left(\frac{K_g K_t}{R_m r_{mp}} \right) \left(-\frac{K_g K_m \dot{x}}{r_{mp}} + V_m \right), \tag{3}$$

where K_t is the motor torque constant, K_g the planetary gearbox gear ratio, K_m the motor back-EMF constant, R_m the motor armature resistance and r_{mp} the motor pinion radius.

Task

1. Read Example 3–5 in Ogata's *Modern Control Engineering* (p. 69 of the printed edition, p. 80 of the PDF): `https://mechfamily-ju.com/storage/images/files/file_17314308026pQTy.pdf`
2. Modify the cart-pole model to match the sign convention of the installed experimental setup. Note that the pendulum angle is measured in the opposite direction to that used in the textbook model.
3. Add in a friction term $-B_f\dot{x}$ for the cart acceleration.
4. The system input is the motor voltage, V_m, which generates the force described by Equation (3). Model the motor dynamics using Equation (3) and integrate them with the cart-pole model to obtain the complete system. Refer to Exercise A-3-9 in the book (p. 95 of the printed edition, p. 106 of the PDF).

The following steps require basic familiarity with Matlab and Simulink. A Matlab introduction can be found here. A basic introduction to Simulink can be found here. It is best to learn by doing, so we recommend directly jumping into the software and playing around for a few minutes.

The cart and pole have rotary encoders attached to their shafts. The rotation angle is then translated into the cart's position and pole's angle. However, this means that there are no absolute cart-position and pole-angle measurements. The zero is assumed to be wherever the cart and pole are when the code starts running.

Please verify that the cart is in the middle of the track and the pole is pointing downwards when launching code on the hardware.

Otherwise, your controller will receive wrong measurements, and implemented safety features may not kick in.

Task

1. Write a Matlab script that simulates only the cart-pole dynamics (neglect the motor dynamics) and run a simulation in Simulink. For now, pick a 10 Hz sinusoid with 5 V amplitude as input signal to the motors.
2. Do the same using the actual system and compare the responses. Therefore, execute the file *system_setup.m* and then run the Simulink model *five_V_ten_Hz_oscillation.slx* by pressing the *Monitor and Tune* button in the *Hardware* tab within Simulink. You may adjust the system parameters to make the response of your simulation more similar to the real system.

Ask your manager to verify your results.

Step 2

PARAMETER ESTIMATION

Unfortunately, our electrical engineers could not figure out the friction coefficient B_f, which is important to model the friction $-B_f\dot{x}_c$ acting on the cart. You can try to estimate its value using a linear regression.

Task

1. Consider the differential equation you derived for $\ddot{x}_c$. Bring the equation into the form $y = \theta\dot{x}_c$ for some $y = y(x, \dot{x}, \alpha, \dot{\alpha}, F_c)$ and constant θ.

Let us first review how linear regression works. Consider a system with noisy outputs of the form

$$y = x^T\theta + \epsilon = x_1\theta_1 + \cdots + x_d\theta_d + \epsilon\,, \tag{4}$$

where $\theta = [\theta_1 \cdots \theta_d]^T \in \mathbb{R}^d$ is an unknown parameter and ϵ is zero-mean gaussian noise with finite variance σ^2. To estimate the parameter θ, we collect a dataset $\{(x^{(i)}, y^{(i)})\}_{i=1}^n$ which corresponds to n observations/measurements performed on the system. Here $x^{(i)} = [x_1^{(i)} \cdots x_d^{(i)}] \in \mathbb{R}^d$ is the input vector and $y^{(i)} \in \mathbb{R}$ is the output/label given the input $x^{(i)}$. In one dimension ($d = 1$), an example is shown in *Figure* 5.

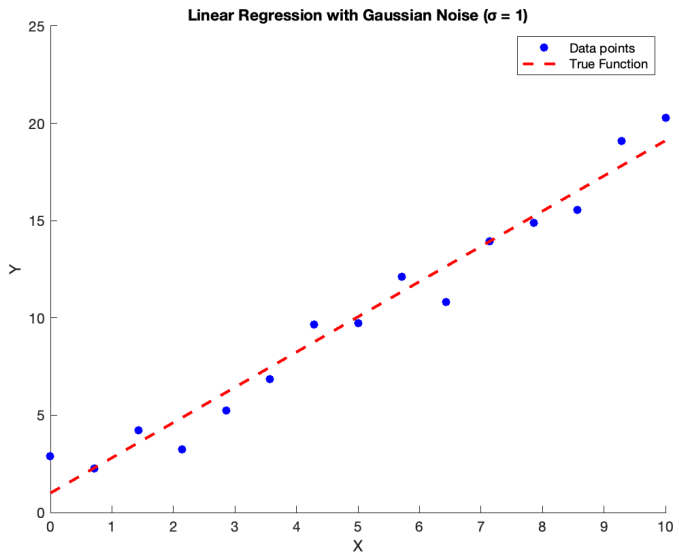


Figure 5: 1D noisy linear system. Each point corresponds to a sample of the dataset. The dotted line corresponds to the *true* function $f(x) = \theta x$.

Our goal is, given the measurements, to find a good estimate $\hat{\theta}$ of the *true* parameter θ. Therefore, we search for the parameter $\hat{\theta}$ which minimizes the squared loss

$$\hat{\theta} = \min_{\tilde{\theta}\in\mathbb{R}^d} \sum_{i=1}^{n} (y^{(i)} - \tilde{\theta}^\top x^{(i)})^2\,. \tag{5}$$

The loss takes into account the distance of each sample to the function $\hat{\theta}^\top x$. We will skip the detailed steps of solving this optimization problem. The parameter that minimizes

the loss function in (5) is given by

$$\hat{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}\,, \tag{6}$$

where

$$\mathbf{X} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \cdots & x_d^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \cdots & x_d^{(2)} \\ & \vdots & & \\ x_1^{(n)} & x_2^{(n)} & \cdots & x_d^{(n)} \end{bmatrix} = \begin{bmatrix} — & x^{(1)} & — \\ — & x^{(2)} & — \\ & \vdots & \\ — & x^{(n)} & — \end{bmatrix} \in \mathbb{R}^{n \times d}$$

and the output vector

$$\mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix} \in \mathbb{R}^n\,.$$

The Simulink model *regression.slx* allows you to move the cart by moving your mouse while recording data for the force **F__c**, for the position **x**, and for the angle **alpha**. The data is automatically saved in the MATLAB workspace.

Task

1. What dimensionality d does the parameter θ have in our problem?
2. Run the model *regression.slx* by opening the file, then pressing the *Monitor and Tune* button in the *Hardware* tab within Simulink. Move the cart back and fourth using your mouse.
3. Estimate the parameter B_f using a least-squares regression. Check with your manager if the values are reasonable.

Step 3

PID CONTROLLER

As a first challenge, the CTO recommends moving the pole into the upright position manually and then stabilizing the pole in the upright position in the center of the track.

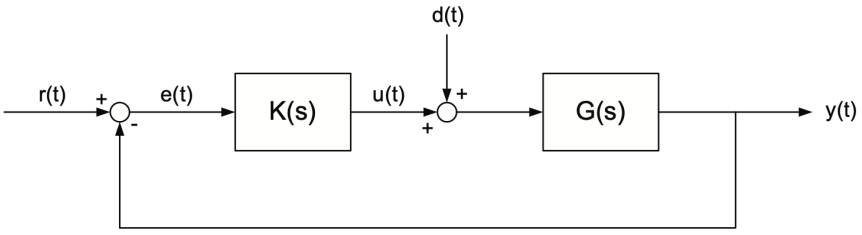


Figure 6: Closed loop system. The output $y(t)$ is compared to the reference signal $r(t)$ and fed back into the system. The system may additionally be affected by disturbances $d(t)$.

A simple controller that we can implement for our system is the PID controller. The feedback structure is depicted in Figure 6, where $G(s)$ are the dynamics of the cart-pole system, and $K(s)$ is the controller. Furthermore, $y(t)$ is the output signal (2-dimensional in our case), $r(t)$ is the reference signal we want to track (upright pole with the cart in the center of the track), $e(t) = r(t) - y(t)$ is the tracking error and $u(t)$ is the control signal produced by the controller. The PID (**P**roportional-**I**ntegral-**D**erivative) control policy is defined as

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \,, \tag{7}$$

where K_p, K_i and K_d are the PID gains that need to be tuned. The effects of the individual terms can be summarized as follows.

- The first term is the **proportional** part. Indeed the input signal is proportional to the tracking error. If we have a large error, we need a large input signal to approach the reference value. Similarly, if the error is small, we need a small input signal, otherwise, the output will overshoot the reference.
- The second term corresponds to the **integral** part. The integral characterizes the error that has been accumulated in the past. This means that the input keeps rising (or decreasing) as long as the error is non-zero. As a consequence, if a controller has an integral part, then in the steady state (as $t \to \infty$) there will be no error.
- The last term is the **derivative** part. The input produced by the controller is proportional to how fast the error is changing. By responding to the rate at which the error changes, it acts as a form of damping, thus reducing oscillations. For example, if the error is increasing quickly, the derivative action produces a control signal to counteract that increase, preventing the error from getting too large and overshooting the reference too much.

In the frequency domain, an integral corresponds to $\frac{1}{s}$ and a derivative to s. So the transfer function of the controller is

$$U(s) = K(s)E(s) = \left(K_p + \frac{K_i}{s} + K_d s\right) E(s) = \left(\frac{K_p s + K_i + K_d s^2}{s}\right) E(s) \,. \tag{8}$$

Unfortunately, the derivative is not implementable as it renders the system non-causal (the degree of the numerator is larger than the degree of the denominator). To overcome this issue, we can multiply the derivative term with a low pass filter with cut-off frequency T_c,

$$G_{LP}(s) = \frac{T_c}{s + T_c} .$$

Then, the controller transfer function equates to

$$K(s) = \left(K_p + \frac{K_i}{s} + K_d \frac{sT_c}{s + T_c} \right) . \tag{9}$$

Since we want to regularize the position and the angle of the cart, we may use two PID controllers and control each variable individually. Our electrical engineers provide a script to implement these controllers.

Task

1. Open the Simulink model *PID.slx* and implement the PID structure for the position and angle. **Hint**: Consider using a lowpass filter for the differentiator to make the controller realizable. Tune the cutoff frequency of the lowpass filter.

2. Tune the parameters until you are satisfied with the system response. **Hints**: Start by tuning K_p, then K_d, and lastly K_i if necessary. Ranges: $K_p \in [0, 120]$, $K_d \in [0, 60]$ and $K_i \in [0, 2]$. The PID output is only passed to the system if the pole is almost upright. Make sure that the integrator of the PID is not integrating errors while the PID is inactive.

3. Once the closed loop system is stable, try to perturb the system by pushing the pole and analyze how the system reacts.

Step 4

STATE FEEDBACK CONTROLLER

We designed PID controllers to independently regularize the cart position and the pole angle. However, both states affect each others dynamics. Intuitively, a controller respecting the interplay of both states may yield a superior performance.

Task

1. Define the state of the system as $x = [x_c\ \alpha\ \dot{x}_c\ \dot{\alpha}]^\top$. Linearize the dynamics of the pole when it is stabilized upright. Therefore, calculate the state-space matrices A, B, C, D such that

$$\dot{x} = Ax + Bu \tag{10}$$
$$y = Cx + Du. \tag{11}$$

2. Check if the system is controllable and observable. **Hint**: Use the functions ctrb() and obsv() to construct the controllability and observability matrices.

Consider the control policy

$$u(t) = -Kx(t)\,, \tag{12}$$

where $x(t) = \begin{pmatrix} x_c & \alpha & \dot{x}_c & \dot{\alpha} \end{pmatrix}^\top$ is the state of the system and $K = [k_1 \cdots\ k_n] \in \mathbb{R}^n$ is an n-dimensional gain. In our specific case, the system is 4-dimensional, i.e., we have four states, hence four parameters for the controller gain. The system dynamics described in (11) become

$$\dot{x}(t) = (A - BK)x(t)\,, \tag{13}$$

where $(A-BK)$ can be interpreted as the matrix of an autonomous system. If all eigenvalues of $(A-BK)$ have negative real part, then the state of the controlled system converges asymptotically to zero. In the following sections, we will see two methods to select the feedback gain K.

Pole Placement[1]

The system behaviour is determined by the poles of the system's transfer function. Controlling the system (11) using the state-feedback controller $u = -Kx$, and assuming $D = 0$ for simplicity, the closed-loop transfer function of the system equates to

$$G_{cl}(s) = C(sI - (A - BK))^{-1}B.$$

Since the demonimator of this transfer function is the characteristic polynomial of the matrix $A-BK$, the poles of $G_{cl}(s)$ equal the eigenvalues of $A-BK$. We therefore aim to place the poles of the system at desired values using the gain K. Fortunately, if (A, B) is controllable, then the eigenvalues of $(A - BK)$ can be freely assigned via the gain vector K.

To ease the decision on where to place the poles, consider the following.

[1] Not the cart-pole pole.

(1) For *stability*, each eigenvalue must have negative real part (left half-plane).

(2) For *robustness*, the poles should be placed far enough from the imaginary axis, otherwise a small deviation in the system parameters could potentially destabilize the system.

(3) To limit *oscillations*, the imaginary part of the poles should remain low, confining them within a damping cone.

(4) The actuator has *limited control power*. In our case, the servo motor has a limited force output, so the controller cannot be overly aggressive. Consequently, the poles should not be placed too far from the imaginary axis. This also reduces the sensitivity to noise.

These conditions restrict the region where we can place the poles, as shown in *Figure* 7.

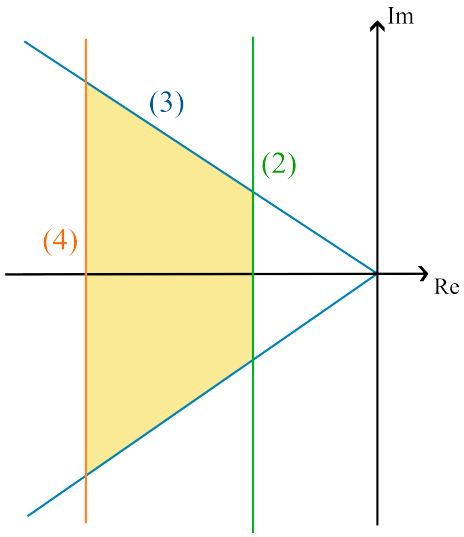


Figure 7: The region of desired pole locations is highlighted in yellow.

Try to design a feedback controller via pole placement.

Task

1. Choose four stable poles and compute the corresponding feedback gain K.
2. Insert your controller into *state_feedback_controller.slx*.
3. Tune the poles until you obtain a satisfactory closed loop system response. **Hint**: Select poles in the range $[-10, 0)$, place them on the real axis and all close to each other.

LQR Controller

Identifying suitable pole locations is challenging, and becomes more and more difficult with increasing state dimensionality. Alternatively, it is possible to describe a good system response via cost functions and, by running an optimization software that computes optimal pole positions, find a controller that minimizes this cost. One particular combination of system and cost functions that yields a simple control low is described by the

Linear Quadratic Regulator (LQR). For linear systems of the form (13) and cost

$$J = \int_0^\infty x^T(\tau)Qx(\tau) + u^T(\tau)Ru(\tau)d\tau \tag{14}$$

the optimal control law turns out to be a linear state feedback controller $u(t) = -Kx(t)$. In (14), $Q = Q^T \geq 0 \in \mathbb{R}^{4\times4}$ is a matrix that gives weights to the states of the system, and $R = R^T > 0 \in \mathbb{R}$ gives weights to the input of the system. The first term in the integral penalizes the state of the system. E.g., if the state is far away from the inverted pole position, then the cost is high. The second term penalizes the control effort. The trade-off between the control effort and control speed is balanced by the choice Q and R.

- Big Q, small R: Penalize the error more than the control input, so the controller is more aggressive and reaches the equilibrium faster. This implies a big controller gain K.
- Small Q, big R: Penalize the control effort more than the error. The input is thus smaller and the equilibrium is reached more slowly. This implies a smaller gain K.

The optimal gain can be computed as

$$K_{\text{lqr}} = R^{-1}B^TP\,, \tag{15}$$

where P is the positive definite solution to the algebraic Riccati equation

$$A^TP + PA + Q = PBR^{-1}B^TP\,. \tag{16}$$

The Riccati equation has a solution if the pair (A, B) is stabilizable. Further, the closed-loop system is guaranteed to be stable if the pair $(A, Q^{\frac{1}{2}})$ is detectable, meaning that all unstable modes are observable. For the tuning of the matrices Q and R it is convenient and intuitive to choose diagonal matrices.

Task

1. Select some values for the matrices Q and R and compute the optimal controller gain K_{lqr}. Then, try the controller on the physical system (same Simulink model as before, just change the gain K) and analyze the response. **Hint**: Use the function lqr().
2. Modify the various elements on the diagonal of the Q matrix to understand how they affect the response of the controlled system.
3. We compute a continuous time controller $u(t) = -Kx(t)$. Is this reasonable? When may this generate issues and how could we overcome them?

Step 5

SWING-UP CONTROLLER

Until now, we have used controllers to stabilize the pole when it was already upright. The goal of the next phase is to find a control policy that automatically brings the pole into the upright region. Once the pole is within a certain upright angle, any of the previously designed controllers can take over to keep the pole upright.

Mouse Reference

To gain some intuition, one may try to manually swing up the pole and take over the job of a Swing-Up controller. Our engineers provide a script that takes the system input (i.e. the servo motor voltage) to be proportional to the velocity of the mouse on the monitor. Note that the mouse can not move beyond the screen, and the velocity is measured as zero if the mouse is stuck at the boundary.

Task

1. Copy your most satisfying implementation of the LQR controller to the new model *swingup_mouse.slx* such that it activates once the angle almost reached the upright position.
2. Run the model on hardware and try to swing the pole up using the mouse. Try to reach the range where the LQR activates.
3. Depending on the pole position, which control inputs did you try to generate? Check your intuition with a manager.

Sinusoidal Reference

You probably moved your mouse back and forth as a function of the pole angle. We can try to emulate the back-and-fourth movement using an input that follows a sinusoidal signal

$$u(t) = F \cdot \sin(\omega t)\,, \tag{17}$$

where F is the amplitude and ω is the frequency (in rad/s). The amplitude and the frequency are key factors in achieving a correct swing-up. If the frequency does not match the resonance frequency of the cart-pole system, then the pole may not swing up.

Task

1. Copy the previous mouse-control implementation in the *swingup_oscillation.slx* model and remove the mouse reference.
2. Use the signal generator block to generate a sinusoidal reference and select the frequency that seems most appropriate to you.
3. For the signal amplitude, use a gain block and start with a value of 1. While running the model, you can then increment this gain online.
4. Tune the frequency and amplitude to achieve swing-up.

Lyapunov Method

Clearly, we want to automize our system and not control it with the mouse. Further, the sinusoidal input is tricky to implement. If the pole is changed to a longer one, then the resonance frequency changes and the sinusoidal input fails to swing the pole up, requiring another extensive manual tuning. A more systematic approach is to analyze the system dynamics and implement a controller that maximizes the energy of the system, achieving a swing-up. Again, copy the previous implementation in the *swingup_lyapunov.slx* model and remove the sinusoidal reference block.

Task

1. Given the total energy

$$E = m_p g r(\cos(\alpha) + 1) + \frac{1}{2}(J_p + m_p r^2)\dot{\alpha}^2 \,,$$

   compute the time derivative.

2. Combined with the differential equation

$$(J_p + m_p r^2)\ddot{\alpha} - m_p g r \sin\alpha = m_p r \ddot{x} \cos\alpha \,,$$

   find a condition on the input that ensures a positive energy increase based on the values of the pole angle, angular velocity and the cart-acceleration.

3. Use a MATLAB function block to implement a controller that increases the total energy at each oscillation. Once upright, switch to another controller that stabilizes the pole in the upright position. **Hint**: To avoid the pole from overshooting, set the input to zero for $|\alpha| < \frac{\pi}{2}$.

Step 6

PLAYGROUND

Good job! You can now swing up and stabilize the pole. To really sell our product on the market, we may want to improve its performance further, or design a show and demonstrate how well our system works. You may continue with one of the following projects.

The Show

Create a nice-looking show. You can use all the controllers you have implemented so far and even add more, especially for the swing-up phase. Some tips:

- A MATLAB function block may help you to code a complex reference signal.
- Use the keyboard keys to allow for user inputs.

Reinforcement Learning

Reinforcement learning (RL) is a type of machine learning technique where an agent learns to make decisions by interacting with an environment, receiving feedback in the form of rewards or penalties. The goal is to maximize cumulative rewards over time by exploring different actions. RL is often used in problems like robotics and autonomous systems, and the cart-pole system is a highly popular benchmark for RL.

Try to implement a reinforcement learning controller by defining the loss and the reward functions. You may use the Reinforcement Learning Toolbox provided by MATLAB.

Iterative Learning Controller

Iterative Learning Control (ILC) is an open-loop control strategy where an agent learns to improve its performance over repeated executions of the same task. It uses information from past iterations to adjust actions and reduce errors, aiming for better precision with each cycle. The control law is

$$u_{t+1} = u_t + Ke_t \,,$$

where t is the time index, K is a constant parameter and the input u_t is being learned epoch after epoch to minimize the tracking error.

Try to implement ILC following the procedure described by MATLAB.

Model Predictive Control

When the system is subject to constraints, such as the track length, Model Predictive Control is a highly popular control approach. Learn more about it here. In fact, it is the industrie's workhorse whenever PID controllers prove insufficient, e.g., for MIMO systems. A quite naive (but effective) implementation of MPC repeatedly samples random input sequences, and predicts the resulting state sequence using a model. This can be quickly done using parallelization on a GPU. At every time-step, only the first input of the most promising sequence is applied; more information can be found here.

Safety Filter

Instead of designing the controller to be safe itself, it is possible to design safety filters for arbitrary control algorithms. One popular technique uses Control Barrier Functions (CBFs). The idea is to compute a function, the CBF, which takes the current system state and desired input as arguments. It then returns whether this state-input combination leads to an inevitable violation of safety constraints in the future. If an input is flagged unsafe, one instead just chooses the closest safe input to the desired desired input.

In fact, this can be done using simple Lyapunov theory: Let the safe set be the sublevel-set $\{x|h(x) \leq 0\}$ of some function $h(x)$. Consider the dynamics $\dot{x}(t) = f(x(t)) + g(x(t))u(t)$, then $\dot{h}(x(t)) = \nabla h(x(t))^\top \dot{x}(t) = \nabla h(x(t))^\top f(x(t)) + \nabla h(x(t))^\top g(x(t))u(t)$, where $\nabla h(x(t))$ is the gradient vector of h with respect to $x(t)$. Let the state $x(t)$ be in the safe set at time zero, $h(x(0)) \leq 0$. Then, the system remains safe, $h(x(t)) \leq 0$ for all $t \geq 0$, as long as $\dot{h}(x(t)) \leq -\gamma h(x(t))$ is always satisfied for all $t \geq 0$ and for some $\gamma > 0$. Now, consider a controller that wants to apply the input $u_{\text{des}}(t)$ at time t. We can simply search for the nearest safe input using the quadratic program $\min_u \frac{1}{2}\|u - u_{\text{des}}(t)\|^2$ subject to $\nabla h(x(t))^\top f(x(t)) + \nabla h(x(t))^\top g(x(t))u(t) \leq -\gamma h(x(t))$.

Design a CBF that ensures the cart never exits the range $[-10, 10]$cm. To keep the derivation simple, approximate the cart as a double integrator system

$$\begin{bmatrix} \dot{x}_c(t) \\ \ddot{x}_c(t) \end{bmatrix} = \begin{bmatrix} \dot{x}_c(t) \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ b \end{bmatrix} u(t).$$

for some $b > 0$. Use the quadprog command in Matlab to solve the quadratic program.

Kalman Filter

The sensors measure the pole angle and the cart position. The respective velocities are obtained using numerical differentiation. However, this strongly amplifies noise. To improve the state-estimation, you can use the provided Simulink block to implement a (extended) Kalman-Filter, or you can code the filter by yourself using a MATLAB function!